PROGRAMMING & CS FUNDAMENTALS

Threads & Processes Visual Guide

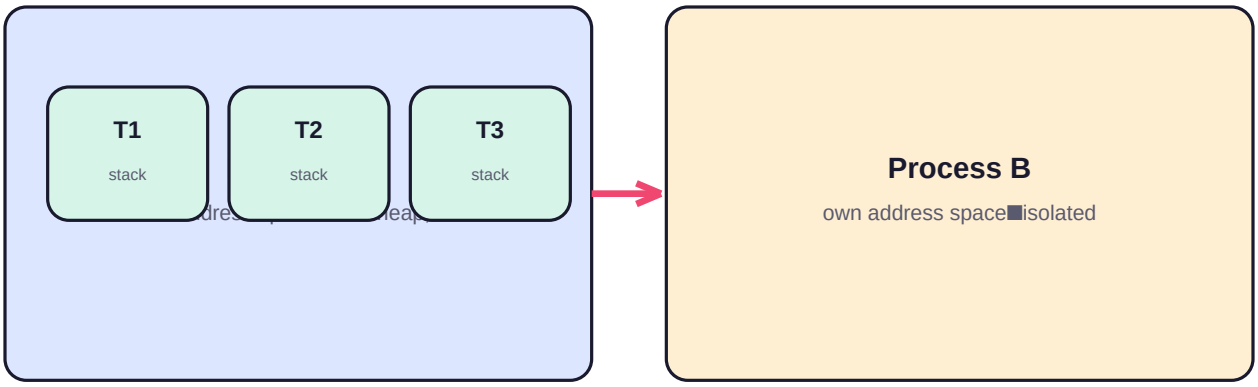
Flows of execution —
address spaces, switching costs, and talking between processes.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

Process vs Thread: Address Spaces

A process is a container; threads are flows inside it.

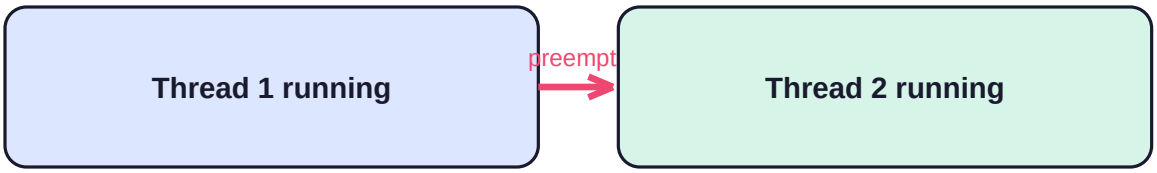


- Process: isolated memory, own file descriptors — crash one, others survive.
- Threads in a process share heap/globals; each has its own stack + registers.
- Threads are cheap to create; processes pay for fork + separate page tables.

STEP 2/6

Context Switching: The Hidden Tax

Swapping what's running costs real time.



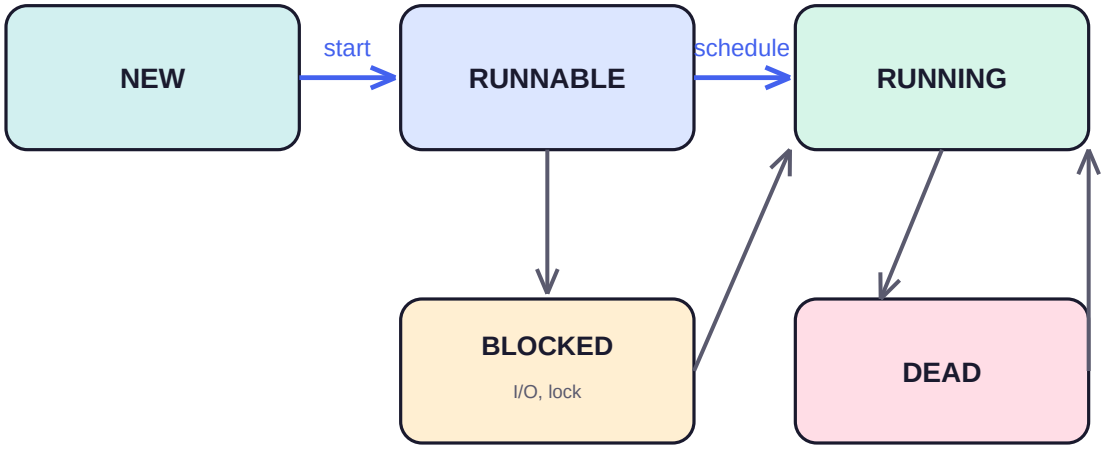

```
# a switch saves/restores:
# registers, program counter, stack pointer, TLB entries, cache warmth
# cost: ~1-10 us thread switch; process switch costs MORE (MMU/TLB flush)
```

- Cache pollution: the new thread finds cold caches — the REAL cost dwarfs register saves.
- Too many threads -> the OS spends more time switching than working (thrashing).
- Rule: ~1-2x CPU cores for CPU work; more only when threads mostly WAIT (I/O).

STEP 3/6

Thread Lifecycle

Born, scheduled, blocked, dead — the OS drives the transitions.



BLOCKED -WAITING-> RUNNABLE: I/O completes, lock frees. DEAD: return or uncaught exception.

Join: one thread waits for another's DEATH. Daemon threads die with the process — never put cleanup-only work in them.

STEP 4/6

IPC: Processes Talking

No shared memory? These are the channels.

Pipes

one-way byte stream, parent-child — e.g. shell pipelines

Message queues

OS-buffered datagrams — e.g. decoupled workers

Shared memory

fastest; needs manual sync — e.g. video frames, big arrays

Sockets

works across machines too — e.g. network services

Signals

tiny async notifications — e.g. Ctrl-C (SIGINT)

Speed ranking: shared memory > pipes/sockets (localhost) > message queues. Pick the weakest channel that does the job.

STEP 5/6

Shared-Memory Pitfalls

Threads share everything — including your bugs.

```
# RACE: two threads, one counter
# T1: read 5 -> add 1 -> write 6
# T2: read 5 -> add 1 -> write 6     # lost update! expected 7

# FIX: lock around read-modify-write, or use atomics
```

- Race condition: outcome depends on interleaving — passes tests, fails at 3 AM.
- False sharing: two threads hammer DIFFERENT fields on one cache line — cache ping-pong.
- Visibility: without sync, writes may sit in a core's cache — others read stale values.
- Thread-safety is a CONTRACT: document it per class ('thread-safe', 'not thread-safe', 'safe if...').

STEP 6/6

Threads vs Processes Chooser

Isolation or sharing — pick your poison.

processes

Need crash isolation — one dies, rest survive

threads

Need max sharing speed — pointer-speed communication

processes

CPU-bound in Python — dodge the GIL

async (or threads)

10k concurrent I/O — threads drown in stacks

processes

Security boundary — separate privilege domains

STEP ✓I✓

Key Takeaways

Threads & Processes — Remember

- Process = isolated address space; threads = flows sharing one space (own stacks).
- Context switches cost cache warmth more than registers — oversubscription thrashes.
- Thread lifecycle: NEW -> RUNNABLE -> RUNNING -> BLOCKED/WAITING -> DEAD; join waits for death.
- IPC ranked by speed: shared memory, pipes/sockets, message queues, signals.
- Shared memory races are interleaving-dependent — lock read-modify-write or use atomics.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev